

CHAPTER 04: Index construction

<https://drive.google.com/file/d/1rJ4uw7h0B6cQVwZ03DrVvyQ-ywJ-43wd/view>

https://drive.google.com/file/d/1oK5tx0pgfXPPCL_N5Bd3A9mjClpEoyTD/view

<https://chatgpt.com/share/699b23e5-3040-800c-8538-adbce84e5246>

Hardware Basics

- When building an IR system, many decisions are based on the characteristics of the computer hardware on which the system runs.
- Access to **data in memory** is **much faster** than access to data on disk
- Disk used for permanent storage [**Disk space** » **RAM size**]
- Memory Access time: ~ 5×10^{-9} seconds (nanoseconds) for a byte
- Disk access time: ~ 2×10^{-8} **seconds**
- we want to **keep as much data** possible in **memory**, especially those data that we need to **access frequently**
- **Caching** → keeping **frequently used disk data** in **main memory**.
- seek time → time taken by the disk head to move to the location where the required data is stored.
 - **5 ms** avg seek time
 - No data is being transferred during seek
- **Chunks** of data will be **read tgt** → shd be **stored contiguously (one continuous chunk)** on disk ⇒ **maximizes data transfer rate!**
- Data non-contiguously stored requires **multiple seeks** ⇒ slower transfer!
- Frequent disk head movement slows down data transfer.

- Operating systems read/write **entire blocks**, not individual bytes.

♦ **Common Block Sizes:** 8 KB , 16 KB, 32 KB, 64 KB, 128KB, 256KB

⚠ **Key Point:**

- Reading **1 byte** takes almost the **same time** as reading the **entire block**.

- **Buffer** → **part of main memory** where a **disk block** is **stored temporarily** during **read/write**

- **Disk-to-memory transfers** are handled by the **system bus**
- NOT handled by the processor (CPU)

✓ **Advantage:** The CPU can process data while disk I/O is happening.

- Idea: Store **compressed data on disk**

Why? ⇒ **Reading + decompressing compressed data** is usually **faster** than **reading uncompressed data**.

- **Fault tolerance** → **expensive**
- **cheaper** to use **many regular machines** rather than one fault tolerant machine.

(2007)

Symbol	Statistic	Value
s	average seek time	$5 \text{ ms} = 5 \times 10^{-3} \text{ s}$
b	transfer time per byte	$0.02 \mu\text{s} = 2 \times 10^{-8} \text{ s}$
	processor's clock rate	10^9 s^{-1}
p	lowlevel operation (e.g., compare & swap a word)	$0.01 \mu\text{s} = 10^{-8} \text{ s}$
	size of main memory	several GB
	size of disk space	1 TB or more

★ basic inverted index:

- The basic steps in constructing a non-positional index as discussed in chapter 1.
- We first make a pass through the collection assembling all term-docID pairs.
- We then sort the pairs with the term as the dominant key and docID as the secondary key.
- Finally, we organize the docIDs for each term into a postings list and compute statistics like term and document frequency.

- For small collections, all this can be done in memory.
- What abt the large collections ?????

Reuters- RCV1 Collection

Table 4.2 Collection statistics for **Reuters-RCV1**. Values are rounded for the computations in this book. The unrounded values are: 806,791 documents, 222 tokens per document, 391,523 (distinct) terms, 6.04 bytes per token with spaces and punctuation, 4.5 bytes per token without spaces and punctuation, 7.5 bytes per term, and 96,969,056 tokens. The numbers in this table correspond to the third line (“case folding”) in Table 5.1 (page 80).

Symbol	Statistic	Value
N	documents	800,000
L_{ave}	avg. # tokens per document	200
M	terms	400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
	tokens	100,000,000

- A **termID** is a unique serial number assigned to each term
- To make index construction more efficient, we **represent terms as termIDs**
- **External sorting** → Sorting method that **uses disk** because data does not fit in memory

› Two ways to build mapping:

- **On-the-fly (single pass)**
- **Two-pass approach (build vocabulary first)**

★ Blocked Sort-Based Indexing [BSBI]

An **external sorting algorithm** for building inverted indexes for large collections.

› BSBI Main Steps:

rithm or BSBI in Figure 4.2. BSBI (i) segments the collection into parts of equal size, (ii) sorts the termID–docID pairs of each part in memory, (iii) stores intermediate sorted results on disk, and (iv) merges all intermediate results into the final index.

Step 1: Segment Collection

- Divide collection into equal-sized blocks
- Each block fits in memory

Step 2: In-Memory Sort

For each block:

1. Parse documents → (termID, docID) pairs
2. Sort pairs in memory

Step 3: Inversion

For each sorted block:

1. Group pairs by termID
2. Create postings list (docIDs only)

✦ Posting = docID

Step 4: Write to Disk

- Each block's inverted index stored on disk

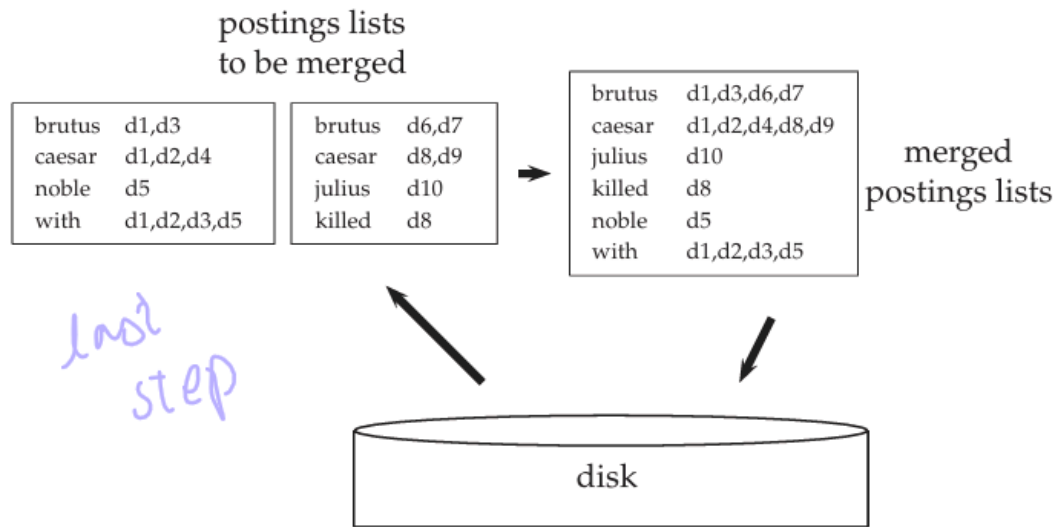
Step 5: Merge Blocks

- Open all block files simultaneously
- Maintain:
 - Read buffers (for blocks)
 - Write buffer (for final index)
- Use priority queue to select lowest termID
- Merge postings lists
- Write merged list to disk

Inversion involves two steps. First, we sort the termID–docID pairs. Next, we collect all termID–docID pairs with the same termID into a postings list, where a *posting* is simply a docID. The result, an inverted index for the block we have just read, is then written to disk. Applying this to Reuters-RCV1 and

In the final step, the algorithm simultaneously merges the ten blocks into one large merged index. An example with two blocks is shown in Figure 4.3, where we use d_i to denote the i^{th} document of the collection. To do the merging, we open all block files simultaneously, and maintain small read buffers for the ten blocks we are reading and a write buffer for the final merged index we are writing. In each iteration, we select the lowest termID that has not been processed yet using a priority queue or a similar data structure. All postings lists for this termID are read and merged, and the merged list is written back to disk. Each read buffer is refilled from its file when necessary.

→ book



› [Algo]

BSBIINDEXCONSTRUCTION()

```

1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4     $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5     $\text{BSBI-INVERT}(block)$ 
6     $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 

```

Figure 4.2 Blocked sort-based indexing. The algorithm stores inverted blocks in files $f_1, \dots, f_n$ and the merged index in f_{merged} .

[1] → n is counter for no. of blocks

[2] → Repeat until the entire collection is read.

[3] → new block being made

[4] →

- Read next chunk of documents.
- Convert documents into **(termID, docID) pairs**.
- Fill memory until block is full.

[5] small inverted index for each block created

- Sort (termID, docID) pairs
- Group same termIDs
- Create postings lists (docIDs per term)

[6]

- Write this inverted block to disk.
 - Store as file $f_1, f_2, \dots, f_n$
 - Free memory for next block.

[7] merge all block indexes.

- Open all block files
- Merge postings lists for same term
- Write final merged index to disk

Result → one complete inverted index.

› Complexity:

- Expensive, cause all blocks are sorted before writing to disk!
- **Sorting dominates**

$$O(T \log T)$$

Where:

- **T = number of termID–docID pairs**

But in practice:

- Parsing documents + Final merging

often dominate actual time.

1 2 Why BSBI is Efficient

- ✓ Avoids loading entire collection in memory
- ✓ Minimizes random disk seeks
- ✓ Uses sequential disk reads
- ✓ Efficient merging

★ Single-pass in-memory indexing [SPIMI]

- Key idea 1: Generate **separate dictionaries** for each block – no need to maintain term-termID mapping across blocks.
- Key idea 2: **Don't sort. Accumulate postings** in postings lists as they occur.

- › With these two ideas we can generate a **complete inverted index for each block**.
- › These **separate indexes** can then be **merged** into **one big index**.
- › dictionary in SPIMI is implemented as a **hash table**.
- › **Every token's term is hashed when it is processed**, even if that term already exists in the dictionary.

[working]

- Processes tokens one by one
- Generates separate dicts for each block
- Builds postings lists directly in memory
- Mem full? Before writing block to disk → sort **terms** in dictionary
- Writes **each block's dictionary + postings to disk**
- **Does NOT use termIDs**
- Does **NOT require sorting** term-doc pairs

It is **more scalable than BSBI** for very large collections.

- **Uses actual terms (strings)**, not termIDs
- Terms added to dictionary by **hashing**
- **No sorting** of term-doc pairs
- We only **sort terms** in the dictionary when **writing a block to disk**.
- **Writes dictionary for each block to disk**
- Starts **fresh dictionary for next block**

› How SPIMI Works

1 Tokens processed one by one

- Stream of term-docID pairs
 - Process sequentially
-

2 When a new term appears

- Add it to dictionary (hash table)
- Create new postings list

3 If term already exists

- Directly append docID to its postings list

✓ Postings lists grow dynamically

4 Dynamic Postings List Growth

Since we don't know postings size beforehand:

- Allocate small space initially
- When full → double its size

(Like dynamic arrays)

5 When Memory Is Full

- Sort terms (not postings)
 - Write:
 - Dictionary
 - Postings liststo disk as one block
 - Clear memory
 - Start new block
-

6 Final Step

- Merge all blocks into one final inverted index
(Same merging step as BSBI)

◆ Major Differences: BSBI vs SPIMI

Feature	BSBI	SPIMI
Uses termIDs	Yes	No
Sorts term-doc pairs	Yes	No
Builds postings directly	No	Yes
Needs global dictionary	Yes	No (per block)
Time complexity	$O(T \log T)$	$O(T)$
Memory usage	Higher	Lower
Scalability	Good	Better

◆ Why SPIMI Is Faster

1 No sorting of term-doc pairs

Sorting is expensive ($T \log T$)

2 Direct postings insertion

No need to store termID repeatedly

3 Better memory usage

No storage of termIDs in intermediate structure

- Each **postings list is dynamic** (size adjusted as it grows) and it is immediately available to collect postings.

› Complexity:

- $O(T)$
- No sorting + all operations at most **linear**

› algo

```

SPIMI-INVERT(token_stream)
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5     if term(token) ∉ dictionary
6        then postings_list = ADDTOdictionary(dictionary, term(token))
7        else postings_list = GETPOSTINGSList(dictionary, term(token))
8     if full(postings_list)
9        then postings_list = DOUBLEPOSTINGSList(dictionary, term(token))
10     ADDTOPOSTINGSList(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKToDISK(sorted_terms, dictionary, output_file)
13 return output_file

```

Figure 4.4 Inversion of a block in single-pass in-memory indexing.

★ Distributed indexing

- For very large collections (like the Web):
 - cannot build index on single server, data is too big for memory and disk of a single system.
- We need **clusters of machines**.

A cluster = many cheap machines working together

♦ Key Idea

- Split the work across many machines using **MapReduce**.
- Final result → a **distributed inverted index**.
- The index can be partitioned:
 - ♦ Term-partitioned (by terms)
 - 👉 Each machine stores postings for **some terms**.
 - **vocabulary is split across machines**.

Machine 1 → terms a–f

Machine 2 → terms g–p

Machine 3 → terms q–z

- ♦ Document-partitioned (by documents)
 - 👉 Each machine stores postings for **some documents**.
 - **documents are split across machines**.
 - Each machine has a mini index for its own docs

Machine 1 → documents d1–d10000

Machine 2 → documents d10001–d20000

This section explains **term-partitioned indexing**.

★ What is MapReduce?

- two phases:
 - 1 Map phase
 - 2 Reduce phase
- It works on **key–value pairs**.
- For indexing:
 - Key = termID
Value = docID

2 types of nodes:

♦ Master Node

- Assigns tasks
- Reassigns if machine fails
- Controls job execution

♦ Worker Nodes

- Execute map or reduce tasks
- Can switch roles

◆ Step-by-Step Process:

1 Splitting Phase

The collection is divided into splits:

- Size: 16MB or 64MB typically
- Balanced workload
- Not too small, not too large
- Partition 1 → terms starting with a–f
- Partition 2 → terms starting with g–p
- Partition 3 → terms starting with q–z

Master assigns splits dynamically.

These groups are called term partitions.

If machine fails → split reassigned.

If we have:

2 Map Phase (Parsing Phase)

- m parsers & n partitions

Machines called parsers do:

We get: $m \times n$ segment files

- Parse documents
- Generate (termID, docID) pairs

Output → written into segment files

Each parser creates multiple segment files
(one per term partition)

◆ Term Partitioning

Keys are divided into ranges:

Example:

- a–f
- g–p
- q–z

Each partition will later go to one inverter.

Important:

All key-value pairs for a partition are stored together.

◆ Data Flow Summary

Collection

→ Splits

→ Parsers (Map phase)

→ Segment files (partitioned by key)

→ Inverters (Reduce phase)

→ Final postings

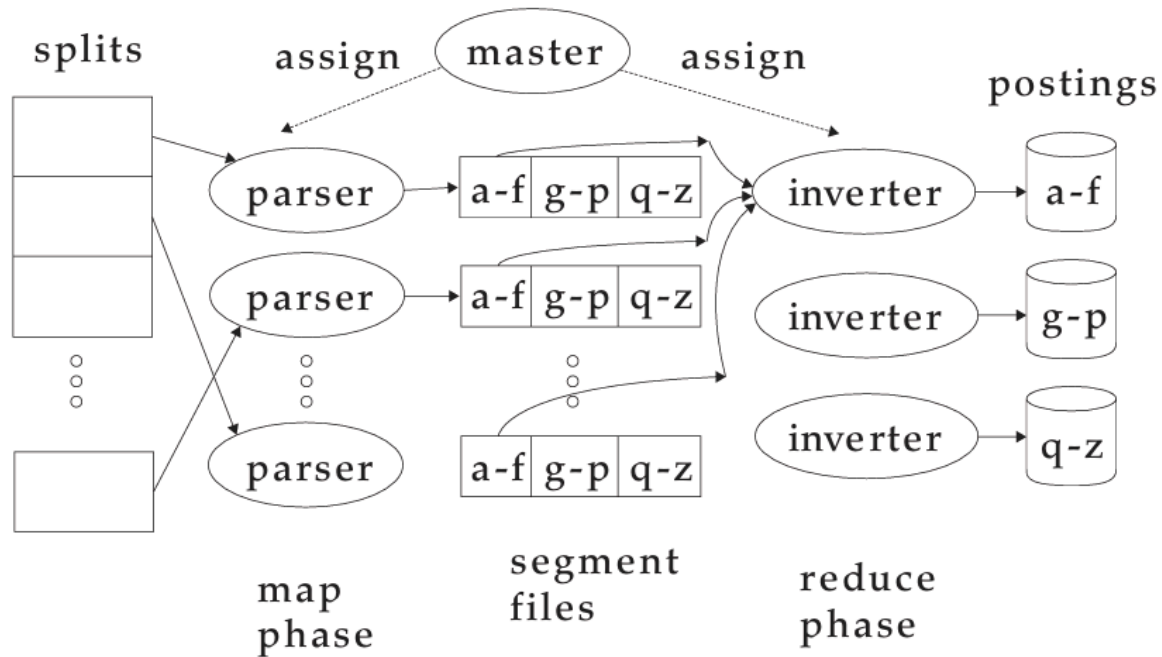
3 Reduce Phase (Inversion Phase)

Machines called inverters:

- Collect all segment files for one partition
- Merge values for each key
- Sort postings
- Write final postings list

Each partition handled by one inverter.





Schema of map and reduce functions

map: input

→ list(k, v)

reduce: ($k, \text{list}(v)$)

→ output

Instantiation of the schema for index construction

map: web collection

→ list(termID, docID)

reduce: ($\langle \text{termID}_1, \text{list}(\text{docID}) \rangle, \langle \text{termID}_2, \text{list}(\text{docID}) \rangle, \dots$)

→ (postings_list₁, postings_list₂, ...)

Example for index construction

map: d_2 : C died. d_1 : C came, C c'ed.

→ ($\langle C, d_2 \rangle, \langle \text{died}, d_2 \rangle, \langle C, d_1 \rangle, \langle \text{came}, d_1 \rangle, \langle C, d_1 \rangle, \langle \text{c'ed}, d_1 \rangle$)

reduce: ($\langle C, (d_2, d_1, d_1) \rangle, \langle \text{died}, (d_2) \rangle, \langle \text{came}, (d_1) \rangle, \langle \text{c'ed}, (d_1) \rangle$)

→ ($\langle C, (d_1:2, d_2:1) \rangle, \langle \text{died}, (d_2:1) \rangle, \langle \text{came}, (d_1:1) \rangle, \langle \text{c'ed}, (d_1:1) \rangle$)

◆ Important Implementation Details

- ✓ Parsers write to local disk
 - ✓ Inverters read sequentially (fast)
 - ✓ Minimal network traffic
 - ✓ Fault tolerance via master reassignment
-

◆ Advantages of Distributed Indexing

- ✓ Scales to huge collections
 - ✓ Fault tolerant
 - ✓ Efficient parallel processing
 - ✓ Uses cheap commodity hardware
 - ✓ Can run other jobs in parallel
-

◆ Complexity & Efficiency

- Map phase → linear in input size
- Reduce phase → merging + sorting postings
- Highly parallel → near-linear scalability



★ Dynamic index:

- Collections are rarely static!
- Documents come in over time and need to be **inserted**.
- Documents are **deleted and modified**.
- So the index must also be updated dynamically.

◆ Simple Solution: Rebuild from Scratch

Idea:

Periodically rebuild the entire index.

Works well if:

- Changes are small
- Delay in indexing new docs is acceptable
- Enough resources exist to build new index while old one is active

Problem:

Not suitable if new documents must appear quickly in search results.

◆ Practical Solution: Main Index + Auxiliary Index

Instead of rebuilding everything, we maintain:

- 1 **Main index** (large, on disk)
- 2 **Auxiliary index** (small, in memory)

> Adding documents:

- New documents go to **auxiliary index (in memory)**

> Searching:

- Query both:
 - Main index
 - Auxiliary index
- Merge results

◆ Handling Deletions

Use an **invalidation bit vector**:

- Each document has a bit
- If **deleted** → bit set to 0
- **Filter** deleted documents during **query processing**

◆ When Auxiliary Gets Large

When auxiliary index size reaches limit n :

→ **Merge it into main index**

◆ Handling Updates

Update = Delete + Insert

- **Mark old** document **deleted**
- **Insert new** version into **auxiliary index**

◆ Storage Design Options

Option 1: One file per postings list ❌

Each term has its own file.

Merging becomes simple:

- Just append postings from auxiliary to main.

But:

- File systems cannot handle millions of small files efficiently.

So this is impractical.

Option 2: One big index file ✅

All postings lists stored in one large file.

More realistic design.

But merging now requires rewriting large portions of the index.

Design	Easy Merge?	Scalable?
One file per term	Yes	❌ No
One big file	Harder	✅ Yes

★ google dance

"Google Dance" was an old term used when:

- Google rebuilt its entire index periodically.
- Rankings would suddenly change.
- Search results looked different for a few days.

It felt like results were "dancing".

◆ How It Relates to Dynamic Indexing

Google's approach (historically):

1 Do incremental indexing

- New pages go into auxiliary postings list
- Quick updates

2 Occasionally rebuild full index

- In background
- Replace old main index

This is exactly what dynamic indexing describes.

★ Cluster vs Grid vs Cloud Computing

◆ 1 Basic Idea

Type	Core Concept	📄
Cluster Computing	Aggregation of resources (many computers act like one system).	
Grid Computing	Segregation of resources (independent systems cooperate for tasks).	
Cloud Computing	Consolidation of resources (services provided on demand via internet).	

	Cluster Computing	Grid Computing	Cloud Computing
Basic Idea	Aggregation of resources.	Segregation of Resources.	Consolidation of Resources.
Running Processes	Same processes run on all computers over the cluster at the same time.	Job is divided into sub-jobs each is assigned to an idle CPU so they all run concurrently.	Depends on service provisioning. Which computer offers a service and provisions it to the requesting clients.
Operating System	All nodes must run the same operating system.	No restriction is made on the operating system.	No restriction is made on the operating system.
Job Execution	Execution depends on job scheduling. So, jobs wait until it's assigned a runtime.	Execution is scalable in a way that moves the execution of a job to an idle processor (node).	Self-Managed.
Suitable for Apps	Cascading tasks. If one task depends on another one.	Not suitable for cascading tasks.	On-demand service provisioning.
Location of nodes	Physically in the same location	Distributed geographically all over the globe.	Location doesn't matter
Homo/Heterogeneity	Homogenous	Heterogeneous	Heterogeneous

	Cluster Computing	Grid Computing	Cloud Computing
Virtualization	None	None	Virtualization is a key
Transparency	Yes	Yes	Yes
Security	High	High, but doesn't reach the level of cluster computing.	Lower than both types.
Interoperability	Yes	Yes	No
Application Domains	industrial sector, research centers, health care, and centers that offer services on the nation-wide level	industrial sector, research centers, health care, and centers that offer services on the nation-wide level	Banking, Insurance, Weather Forecasting, Space Exploration, Business, IaaS, PaaS, SaaS
Implementation	Easy	Difficult	Difficult – need to be done by the host.
Management	Easy	Difficult	Difficult
Resource Management	Centralized (locally)	Distributed	Both centralized and distributed.
Internet	No internet access is required	Required	Required



1. What are the problems with traditional inverted index approach introduced in chapter 1? Illustrate them.

- Assumes entire collection fits in memory.
 - Sorting all (term, docID) pairs requires large RAM.
 - Not scalable for large collections (e.g., 100M tokens).
 - Example: Reuters-RCV1 would need ~0.8GB just for termID-docID pairs.
-

2. How Block-Sort Based Indexing (BSBI) gain advantages over traditional inverted indexing approach?

- Processes data in blocks that fit in memory.
 - Uses external sorting (disk-based).
 - Reduces memory requirement
 - Scales to very large collections.
-

3. What are some of the drawbacks of BSBI?

- Requires sorting of termID-docID pairs ($O(T \log T)$).
 - Needs global **term** → **termID** mapping in memory.
 - Multiple disk I/O operations.
 - Less memory efficient than SPIMI.
-

4. How would you create the dictionary in BSBI on the fly to avoid an extra pass through the data?

Answer:

- Use a **hash table** while parsing documents.
 - When a new term appears, assign it a new termID immediately.
 - Store mapping dynamically during indexing.
 - Avoid separate vocabulary-building pass.
-

5. Explain how SPIMI gains advantages over BSBI.

Answer:

- No sorting of term-doc pairs.
 - **Builds postings lists directly.**
 - Uses terms instead of termIDs.
 - Lower memory usage.
 - Time complexity **$O(T)$** .
-

6. Compare SPIMI and BSBI in terms of time complexities.

Algorithm	Time Complexity
BSBI	$O(T \log T)$
SPIMI	$O(T)$

SPIMI is faster because it avoids sorting.

7. Compare the differences between BSBI and SPIMI.

Feature	BSBI	SPIMI
Uses termIDs	Yes	No
Sorting required	Yes	No
Dictionary	Global	Per block
Memory usage	Higher	Lower
Time complexity	$O(T \log T)$	$O(T)$
Scalability	Good	Better